

ChurchServers Management System (CSMS)

Unified Technical Plan: Results + Attendance

Version: 1.0
Date: December 27, 2025
Prepared for: Church Leadership + Development Team
Deadline: 3 weeks from now

Executive Summary

The Opportunity

بتوحيد تطبيقي النتائج والحضور في **نظام واحد شامل**، نحل المشاكل الأمنية والعملية ونبني منصة قابلة للتوسع للمستقبل.

Current State: Two separate projects, security gaps, distribution nightmare
Target State: One unified, offline-first, secure mobile app + scalable backend
Timeline: 20–26 weeks (5–6 months) | non-rushed, quality-focused
Investment: Technical expertise + Firebase free tier (\$0–\$300/month at scale)

Part 1: Real-World Problem Analysis

1.1 Critical Security Issues

Problem 1: Student Results Exposed (🔴 CRITICAL)

Current Implementation:

- User enters ID number only → no authentication
- Anyone can guess colleague's ID → view their grades
- No audit trail → no way to know who accessed what

Real-world impact:

- Student privacy violation
- Can't prove data integrity
- Violates church data governance

Solution: Username + Password + Session management + Audit logs

Problem 2: Attendance Single Points of Failure (🔴 HIGH)

Current Implementation:

- Only one admin can mark attendance
- If that person is absent → whole system breaks
- No distributed access control

Real-world impact:

- Service disruption risk
- Unfair dependency on one person
- Can't scale to new servants

Solution: Role-based access + multi-servant capability

Problem 3: Distribution is Manual Hell (🔴 HIGH)

Current Implementation:

- APK distributed manually to Android devices
- Every update = re-distribute to all
- Version fragmentation (some phones on old version)
- Updates fail silently

Real-world impact:

- Features get stuck on old versions
- Security patches don't reach everyone
- Hours wasted on manual distribution

Solution: Google Play auto-distribution + Firebase Remote Config for feature flags

Problem 4: Offline Doesn't Really Work (🔴 HIGH)

Current Implementation:

- Mobile app built but relies on network
- Church WiFi is unstable
- Attendance data gets lost if connection drops

Real-world impact:

- Data loss
- User frustration
- Unreliable system

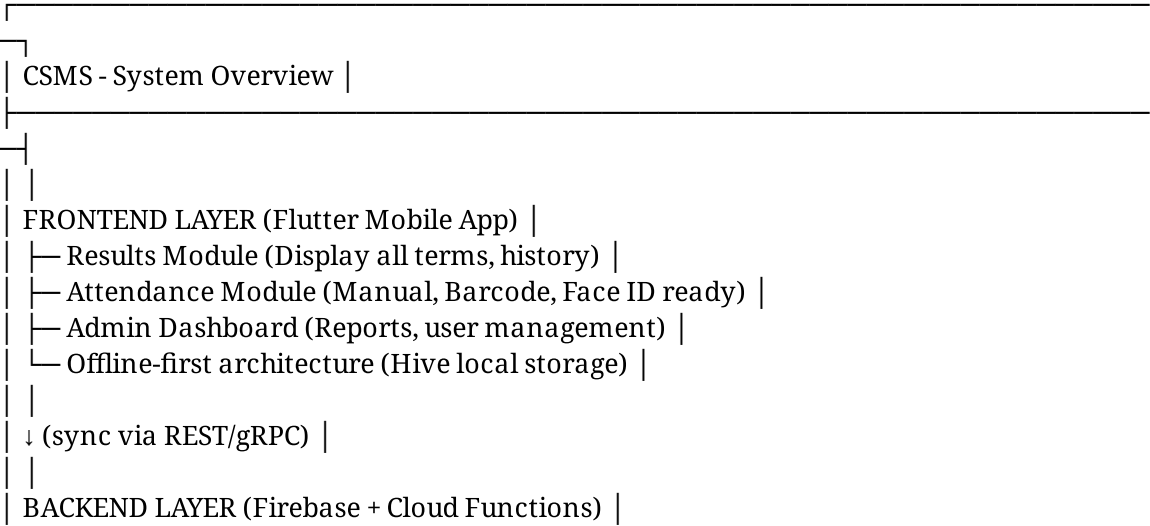
Solution: Proper offline-first architecture with conflict-free sync

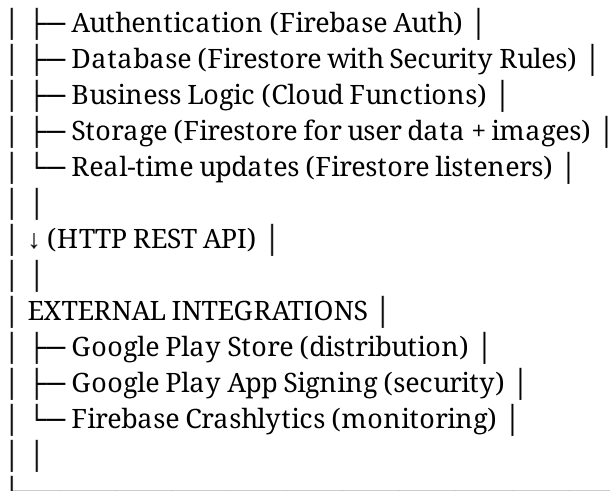
1.2 Architectural Problems

Problem	Current State	Impact	Solution
No unified data model	Two separate databases	Manual sync nightmare	Firestore as single source of truth
No session management	Stateless (ID lookup only)	Security gap	JWT + Firebase Auth
No role-based access	Admin vs user unclear	Permission creep	Firestore Security Rules + role matrix
No sync strategy	No offline support	Data loss	Hive (local) + Firestore (cloud) with conflict resolution
No error handling	App crashes silently	Unknown failures	Comprehensive error handling + crash reporting
No analytics	Can't see usage patterns	Blind deployment	Firebase Analytics built-in

Part 2: Proposed Architecture (Unified System)

2.1 High-Level Design





2.2 Core Modules Architecture

Module 1: Authentication & User Management

Responsibility: Secure login, role-based access, session management

Components:

- Firebase Authentication (email/password)
- Custom JWT for stateless API calls
- Servant profiles (name, role, permissions, photo)
- Role matrix: Admin / Servant / Teacher / Viewer

How it works:

1. Servant enters username + password
2. Firebase validates credentials
3. JWT issued with role + permissions
4. App stores JWT locally (encrypted Hive)
5. All requests include JWT in headers
6. Server validates JWT before responding

Security:

- Passwords hashed (Firebase default bcrypt)
- JWT expires every 24 hours (refresh token strategy)
- No passwords stored locally
- All sensitive data encrypted at rest

Interface:

```
// Login flow
final user = await FirebaseAuth.instance.signInWithEmailAndPassword(
  email: username + "@church.local",
  password: password,
);
final idToken = await user.getIdToken();
// Store idToken in secure storage (flutter_secure_storage)
```

Module 2: Results Management (Redesigned)

Current Problem: Single term, no history, no security

New Design: Multi-term history, timeline view, per-servant access control

Components:

- Results repository (Firestore: results/{servantId}/terms)
- Results UI (timeline, term selector, filters)
- Admin panel (upload results, manage terms)
- Audit logs (who viewed what, when)

Data Model:

```
results/  
├── {servantId}/  
│   ├── terms/  
│   │   ├── term_1/  
│   │   │   ├── date: "2024-01-15"  
│   │   │   ├── scores: {sub1: 95, sub2: 88...}  
│   │   │   └── average: 91.5  
│   │   ├── term_2/...  
│   │   └── term_3/...  
│   ├── history: [] // For audit  
│   └── lastUpdated: timestamp  
└── metadata/  
    ├── terms: ["Fall 2023", "Spring 2024", "Fall 2024"]  
    └── subjects: ["Scripture", "Service", "Character", ...]
```

Access Control (Firestore Rules):

```
// Servant can only see their own results  
match /results/{servantId}/{document=**} {  
  allow read: if request.auth.uid == servantId;  
  allow write: if request.auth.token.role == 'admin';  
}
```

How it works:

1. Servant opens app → sees their term history (local Hive first)
2. If online → fetch latest from Firestore + merge
3. Servant selects term → view all scores + average
4. Servant swipes → see previous terms (visual timeline)
5. Admin uploads new results → automatic sync to all servants' phones

Module 3: Attendance Management (Complete Redesign)

Current Problem: Single admin, complex Face ID, distribution nightmare

New Design: Offline-first, multi-servant capable, progressive complexity

Phases:

- **Phase 1 (v1.0):** Manual selection + Barcode scanning
- **Phase 2 (v1.5):** Face ID optional (if trained model ready)
- **Phase 3 (future):** Hybrid (Face ID primary, manual fallback)

Components:

- Attendance service (Firestore: attendance/{serviceId}/records)
- Attendance UI (servant list, quick mark, batch import)
- Barcode scanner integration (flutter_barcode_scanner)
- Admin reports (attendance rates, trends, export)
- Offline sync (Hive → Firestore when online)

Data Model:

```
attendance/  
├── {serviceId}/  
│   ├── date: "2024-12-27"  
│   ├── service_type: "Morning Service"  
│   ├── servant_count: 24  
│   ├── recorded_by: {uid: "admin1", name: "Father John"}  
│   └── records/  
│       ├── {servantId}/  
│       │   ├── status: "present" | "absent" | "late"  
│       │   ├── timestamp: 1703692800  
│       │   ├── method: "manual" | "barcode" | "face_id"  
│       │   └── verified_by: {uid: "admin1"}  
│       └── ...more servants...  
└── synced_at: timestamp  
    └── metadata: {offline_recorded: false}  
└── local_cache // On device, synced later  
    └── {serviceId}/  
        └── records: {...}
```

How it works (v1.0 - Manual + Barcode):

1. **At home (prep):** Admin opens app, selects today's service, offline mode loads servant list from Hive
2. **At church (during service):** Multiple servants can open app (no single admin)
3. **Mark attendance - Method 1 (Manual):**
 - Servant opens app → sees alphabetical list
 - Taps servant name → mark present/absent
 - Local Hive updates immediately (instant feedback)
4. **Mark attendance - Method 2 (Barcode):**
 - Servant opens barcode scanner
 - Scans servant's ID card → automatic lookup + mark present
 - Works offline (barcode = local ID lookup)
5. **After service (sync):** When connected to WiFi → all records sync to Firestore
6. **Conflict resolution:** If two devices mark same servant differently:
 - Use "last-write-wins" with manual override option
 - Admin can reconcile conflicts in report view

Offline-First Implementation:

```
// All reads from local Hive first  
final cachedRecords = await hiveBox.get('service_${serviceId}');  
  
// If online, fetch & merge from Firestore  
if (isConnected) {
```

```

final firebaseRecords = await firestore.collection('attendance').doc(serviceId).get();
final merged = mergeWithConflictResolution(cachedRecords, firebaseRecords);
await hiveBox.put('service_${serviceId}', merged);
}

// All writes go to local Hive immediately
await hiveBox.put('attendance_{servantId}', 'present');

// Background sync when online
syncManager.addJob(() async {
await firestore.collection('attendance').doc(serviceId).update(localData);
});

```

Module 4: Admin Dashboard (New)

Components:

- User management (create/edit servants, assign roles)
- Results upload (bulk CSV import)
- Attendance reports (who's present, trends, export)
- Sync status (what's cached locally, what's synced)
- Audit logs (who did what, when)

Access Control:

```

// Only admin can access admin dashboard
match /admin/{document=**} {
allow read, write: if request.auth.token.role == 'admin';
}

```

2.3 Data Flow (How Everything Communicates)

Flow 1: Servant Logs In

1. App (offline) → Show login screen
2. Servant enters: username + password
3. App → Firebase Auth (if online)
4. Firebase → Returns idToken + refreshToken
5. App → Hive (store token securely)
6. App → Load home screen (sync in background)

Flow 2: View Results (Multi-Term History)

1. Servant opens "Results" tab
2. App → Check Hive for cached terms (instant load)
3. If online:
 - a. App → Firestore query: results/{servantId}/terms
 - b. Firestore → Returns all terms (with permission check)
 - c. App → Merge with local cache
 - d. Hive → Update cache
4. UI renders timeline (newest → oldest)

Flow 3: Mark Attendance (Offline → Sync)

At Church (Offline):

1. Servant opens "Attendance" tab
2. App → Load servant list from Hive (instant)
3. Servant taps name → mark "present"
4. App → Hive (update immediately)
5. UI shows checkmark ✓ (instant feedback)

Later (When Connected):

6. Sync manager wakes up
7. App → Compare local Hive with Firestore
8. Firestore → Apply updates (if no conflicts)
9. Firestore → Log to audit trail
10. Hive → Mark as "synced"
11. App → Notify user "Attendance synced ✓"

Flow 4: Admin Uploads New Results

1. Admin opens "Results Management"
2. Admin selects CSV file or enters manually
3. Admin reviews preview (all servants + scores)
4. Admin hits "Publish Term"
5. App → Cloud Function (validate data)
6. Cloud Function → Write to Firestore: results/{servantId}/...
7. Firestore Realtime → Triggers all connected servants' apps
8. Each servant's app → Hive (update cache automatically)
9. Notification → Each servant gets "New results available"

Part 3: Technology Stack (Decision Matrix)

Component	Technology	Why	Alternative	Rejected Because
Frontend	Flutter	Single codebase (iOS + Android)	React Native, Swift/Kotlin	Need fast delivery, cross-platform
Mobile Storage	Hive	Offline-first, no SQL, fast	SQLite, Realm	Simpler, no schema migration needed
Backend DB	Firestore	Real-time, offline support, security rules	Firebase Realtime DB, Supabase	Better scalability + built-in sync
Authentication	Firebase Auth	Stateless, secure, zero-config	JWT custom, Auth0	Firebase integrates with Firestore rules
Real-time Sync	Firestore Listeners	Automatic, conflict-free	REST polling, Socket.io	Built-in, battle-tested
Distribution	Google Play Store	Auto-updates, no manual APK	APK manual, Firebase App Distribution	Professional, automatic security patches
Barcode Scanning	flutter_barcode_scanner	Simple, reliable	google_mlkit, qr_mobile_vision	Production-ready, minimal setup
Local Encryption	flutter_secure_storage	Secure JWT/tokens locally	Shared preferences, Hive alone	Prevents token theft if phone is stolen

Component	Technology	Why	Alternative	Rejected Because
Monitoring	Firebase Crashlytics	Automatic error tracking	Sentry, DataDog	Free, built-in, minimal config
File Uploads	Firebase Storage	Integrated with Firestore	AWS S3, Cloud Storage	Easy integration, CORS handled

Part 4: Unified System Architecture

4.1 Why Merge into One App?

✓ Advantages

Aspect	Single App	Two Apps
User Experience	Servants see both results + attendance in one place	Switch between apps = context loss
Offline Sync	One sync engine, one source of truth	Two separate sync logic = conflicts
Updates	One APK to maintain + distribute	Two APKs = version fragmentation
Data Consistency	One database, one schema	Manual sync between two systems
Permissions	Granular per-feature (some see results, some mark attendance)	App-level permissions (all-or-nothing)
Development	Shared components (auth, storage, sync)	Duplicate code, maintenance nightmare
Rollout	Single deployment to Google Play	Two separate deployments

□ Recommendation

Unified Single App because:

- Servants are the same people (results + attendance both matter)
- Sync complexity explodes with two systems
- Distribution headache becomes trivial
- Future features (notifications, analytics) need both modules together

4.2 App Structure (Clean Architecture)

```
lib/
├── main.dart # App entry, routes
├── core/
│   ├── constants/
│   │   ├── app_strings.dart # UI text (Arabic support)
│   │   ├── app_colors.dart # Theme colors
│   │   └── api_endpoints.dart # Firebase paths
│   ├── errors/
│   │   ├── exceptions.dart # Custom exceptions
│   │   └── failure.dart # Failure models
│   ├── services/
│   │   ├── firebase_service.dart # Firestore init
│   │   ├── auth_service.dart # Firebase Auth wrapper
│   │   ├── sync_service.dart # Offline sync manager
│   │   └── storage_service.dart # Hive wrapper
│   ├── utils/
│   │   ├── date_utils.dart
│   │   └── validators.dart
│
├── features/
│   ├── auth/
│   │   ├── data/
│   │   │   ├── datasources/
│   │   │   │   ├── auth_local_datasource.dart # Hive (cached user)
│   │   │   │   └── auth_remote_datasource.dart # Firebase Auth
│   │   │   └── models/
│   │   │       └── servant_model.dart # Servant {id, name, role, email}
│   │   │   └── repositories/
│   │   │       └── auth_repository_impl.dart # Combines local + remote
│   │   └── domain/
│   │       ├── entities/
│   │       │   └── servant.dart # Pure domain entity
│   │       └── usecases/
│   │           ├── login_usecase.dart # (username, password) → Servant
│   │           └── logout_usecase.dart
│   └── presentation/
│       ├── pages/
│       │   └── login_page.dart # UI
│       └── bloc/
│           └── auth_bloc.dart # State management
```

```

| | | | | auth_event.dart
| | | | | auth_state.dart
| | | | | widgets/
| | | | | login_form_widget.dart
|
| | | | | results/
| | | | | | | data/
| | | | | | | | | datasources/
| | | | | | | | | | | results_local_datasource.dart # Hive
| | | | | | | | | | | results_remote_datasource.dart # Firestore
| | | | | | | | | models/
| | | | | | | | | | | term_model.dart # {id, name, date, scores}
| | | | | | | | | | | result_model.dart # {servantId, terms[]}
| | | | | | | | | repositories/
| | | | | | | | | | | results_repository_impl.dart # Merge local + remote
| | | | | | | domain/
| | | | | | | | | entities/
| | | | | | | | | | | term.dart
| | | | | | | | | usecases/
| | | | | | | | | | | get_all_terms_usecase.dart # → List<Term>
| | | | | | | | | | | get_term_details_usecase.dart
| | | | | | | presentation/
| | | | | | | | | pages/
| | | | | | | | | | | results_list_page.dart
| | | | | | | | | | | term_detail_page.dart
| | | | | | | | | bloc/
| | | | | | | | | | | results_bloc.dart
| | | | | | | | | | | results_event.dart
| | | | | | | | | | | results_state.dart
| | | | | | | | | widgets/
| | | | | | | | | | | term_card_widget.dart
| | | | | | | | | | | score_chart_widget.dart
|
| | | | | attendance/
| | | | | | | data/
| | | | | | | | | datasources/
| | | | | | | | | | | attendance_local_datasource.dart # Hive (offline cache)
| | | | | | | | | | | attendance_remote_datasource.dart # Firestore
| | | | | | | | | models/
| | | | | | | | | | | service_model.dart # {id, date, type, records[]}
| | | | | | | | | | | attendance_record_model.dart # {servantId, status, method, timestamp}
| | | | | | | | | | | servant_model.dart # For attendance (lite version)
| | | | | | | | | repositories/
| | | | | | | | | | | attendance_repository_impl.dart # Handles offline + sync
| | | | | | | | | domain/
| | | | | | | | | | | entities/
| | | | | | | | | | | | | service.dart
| | | | | | | | | | | | | attendance_record.dart
| | | | | | | | | | usecases/
| | | | | | | | | | | mark_present_usecase.dart # → void (instant local update)
| | | | | | | | | | | get_today_servants_usecase.dart # → List<Servant>

```

```

| | | | | sync_attendance_usecase.dart # → void (when online)
| | | | | presentation/
| | | | | pages/
| | | | | | attendance_list_page.dart
| | | | | | barcode_scanner_page.dart
| | | | | | attendance_report_page.dart
| | | | | bloc/
| | | | | | attendance_bloc.dart
| | | | | | attendance_event.dart
| | | | | | attendance_state.dart
| | | | | widgets/
| | | | | | servant_tile_widget.dart
| | | | | | mark_button_widget.dart
| | | | |
| | | | | admin/
| | | | | presentation/
| | | | | | pages/
| | | | | | | admin_dashboard_page.dart
| | | | | | | user_management_page.dart
| | | | | | | results_upload_page.dart
| | | | | | | sync_status_page.dart
| | | | | | bloc/
| | | | | | | admin_bloc.dart
| | | | | | widgets/
| | | | | | | upload_form_widget.dart
| | | | |
| | | | | widgets/
| | | | | | app_scaffold.dart # Main layout (navbar + body)
| | | | | | sync_indicator_widget.dart # Shows "Online/Offline" + sync status
| | | | | | error_dialog_widget.dart # Unified error handling

```

Part 5: Implementation Phases (20–26 Weeks)

Phase 1: Foundation & Security (Weeks 1–8)

Goal: Secure authentication + offline-first setup + data migration

Tasks:

1. Firebase Project Setup (Week 1)

- Create Firebase project (console.firebase.google.com)
- Enable: Authentication, Firestore, Cloud Storage, Functions
- Set up Firestore security rules (no public access)
- Create service accounts for admin scripts

2. Flutter Project Init (Week 1–2)

- Create Flutter project with clean architecture
- Add dependencies: `firebase_core`, `firebase_auth`, `cloud_firestore`, `hive`, `flutter_secure_storage`
- Set up folder structure (as above)
- Configure Firebase in app

3. Authentication Module (Week 2–4)

- Firebase Auth setup (email/password provider)
- Custom user model: Servant { id, email, username, name, role, department }
- Login use case + repository
- Secure token storage (JWT in flutter_secure_storage)
- Logout + session expiry
- Tests for auth (happy path + edge cases)
- 4. Local Storage Setup (Week 3–4)**
 - Hive boxes: servants, results, attendance, sync_meta
 - Encryption at rest (Hive encryption adapter)
 - Migration logic (old → new format)
 - Clear vs. keep data on logout
- 5. Firestore Schema + Security (Week 4–5)**
 - Firestore collections:
 - servants/: user profiles + roles
 - results/: per-servant term history
 - attendance/: per-service records
 - audit_logs/: who did what, when
 - Security rules (role-based):
 - Servant can read own results + attendance
 - Admin can read/write all
 - Indexes (for queries like "attendance by date range")
- 6. Data Migration Plan (Week 5–6)**
 - Export data from old web app (students_results.git)
 - Transform old data → new Firestore schema
 - One-time migration script (Cloud Function)
 - Validation + checksums
- 7. Sync Engine (Core) (Week 6–8)**
 - Offline detection (connectivity_plus)
 - Hive as source of truth when offline
 - Background sync job (when online):
 - Diff local vs. Firestore
 - Upload local changes
 - Download remote changes
 - Conflict resolution (last-write-wins + manual override)
 - Tests for sync scenarios (online/offline transitions)
- 8. Error Handling + Logging (Week 7–8)**
 - Custom exception hierarchy
 - Firebase Crashlytics integration
 - In-app error dialogs
 - Debug logging (turn off in production)

Deliverable:

- Secure login working (online + offline detection)
 - Local storage + sync engine functional
 - Old data migrated to Firestore
 - Tests for auth + sync
-

Phase 2: Core Features (Weeks 9–18)

Goal: Results module + Attendance module (manual + barcode) fully functional

2A: Results Module (Weeks 9–12)

Tasks:

1. Results Repository + Use Cases (Week 9)

- GetAllTermsUseCase: → List<Term> (from Hive if offline, Firestore if online)
- GetTermDetailsUseCase: → Term with all scores
- CacheResultsUseCase: Local Hive updates
- Offline merge logic

2. Results UI (Week 10–11)

- Results list page (all terms, newest first)
- Term detail page (all subjects + scores, average calculation)
- Timeline visualization (when did scores improve/decline)
- Search + filter by term name
- Accessibility (dark mode, large text)

3. Results BLoC (Week 11)

- State management: Loading / Loaded / Error
- Events: FetchTerms / SelectTerm / RefreshResults
- Real-time listeners (Firestore → auto-update UI when new results available)

4. Tests (Week 12)

- Unit: repository merge logic, use cases
- Widget: results list rendering, date sorting
- Integration: offline → online transition

Deliverable:

- Servants can view all their term history
- Works offline (uses Hive cache)
- Syncs automatically when online

2B: Attendance Module - Phase 1 (Weeks 13–18)

Tasks:

1. Attendance Repository + Use Cases (Week 13)

- GetTodayServantListUseCase: → List<Servant>
- MarkPresentUseCase: → void (write to Hive immediately)
- GetAttendanceRecordsUseCase: → List<AttendanceRecord>
- Conflict resolution logic

2. Barcode Scanner Integration (Week 13–14)

- flutter_barcode_scanner setup
- Barcode format: Servant ID card number
- Parse barcode → lookup servant ID locally → mark present
- Barcode validation (check digit, length)
- Error handling (invalid barcode, servant not found)

3. Attendance UI - Manual (Week 14–15)

- Service type selector (Morning / Afternoon / Special)
- Servant list (alphabetical, filterable)
- Tap to mark: Present / Absent / Late

- Visual feedback (color change, checkmark)
- Quick actions (Mark All Present, Clear All)
- 4. Attendance UI - Barcode** (Week 15–16)
 - Barcode scanner button → open camera
 - Real-time scan feedback
 - Auto-mark after scan
 - Sound notification (optional)
- 5. Offline Sync for Attendance** (Week 16–17)
 - Mark attendance locally (instant Hive update)
 - Background sync when online
 - Conflict resolution (if two devices mark same person differently)
 - Audit trail (who marked, when, from which device)
- 6. Attendance Report** (Week 17)
 - View attendance by date
 - Export to CSV
 - Attendance rate (%) per servant
- 7. Tests** (Week 18)
 - Unit: barcode parsing, conflict resolution
 - Widget: servant list, mark UI, barcode scanner
 - Integration: offline mark → online sync

Deliverable:

- Servants (not just admin) can mark attendance
- Manual + barcode scanning functional
- Works 100% offline
- Syncs automatically when online

Phase 3: Admin Features + Scale (Weeks 19–22)

Goal: Admin dashboard + results upload + Google Play deployment

Tasks:

- 1. Admin Role + Permissions** (Week 19)
 - Firestore security rules for admin-only operations
 - Admin flag in servant profile
 - Admin-only pages (restricted at UI level)
- 2. Results Upload** (Week 19–20)
 - CSV import: (servantId, subject1_score, subject2_score, ...)
 - Manual entry form (alternative to CSV)
 - Preview before publish (show all servants + scores)
 - Publish → writes to Firestore
 - Notification to all servants (real-time Firestore listener)
- 3. User Management Dashboard** (Week 20)
 - Create servant account (email → auto-username, send password via SMS/email)
 - Edit servant details (name, department, role)
 - Deactivate/reactivate servant
 - Reset password
 - Audit log (who created/edited user)

4. **Sync Status Dashboard** (Week 20–21)
 - Show: what's locally cached, what's been synced
 - Manual "Sync Now" button
 - Conflict resolution UI (for manual intervention)
 - Device status (online/offline indicator)
5. **Reports & Analytics** (Week 21)
 - Attendance rate by servant (%)
 - Attendance trends (week-over-week)
 - Results trends (avg score by term)
 - Export to PDF/CSV
6. **Google Play Preparation** (Week 21–22)
 - App signing setup (generate keystore)
 - Build release APK
 - Create Google Play Console account
 - Write app description + screenshots + privacy policy
 - Submit for review (usually 2–4 hours)
 - Internal testing track first

Deliverable:

- Admin can upload results + manage users
 - App on Google Play (auto-updates)
 - Reports generated
-

Phase 4: Advanced & Future (Weeks 23–26 + Beyond)

Goal: Polish + optional advanced features

Tasks:

1. **Face ID Setup (Optional, v1.5+)** (Week 23–24)
 - Requires ML model training (outside scope for now)
 - Integration point: `mark_face_id_usecase.dart`
 - Fallback to manual if Face ID fails
2. **Notifications** (Week 24)
 - Firebase Cloud Messaging (FCM)
 - New results available → notify servants
 - Attendance sync complete → notify admins
 - In-app + push notifications
3. **Analytics Dashboard** (Week 25)
 - Firebase Analytics events (e.g., "MarkAttendanceEvent")
 - Dashboard: daily active users, feature usage, crashes
 - Identify bottlenecks
4. **Performance Optimization** (Week 25–26)
 - Profile app (performance monitor)
 - Lazy load results/attendance lists
 - Pagination for large datasets
 - Caching strategy refinement
5. **Security Audit** (Week 26)
 - Penetration testing (self + code review)
 - Firestore rules audit

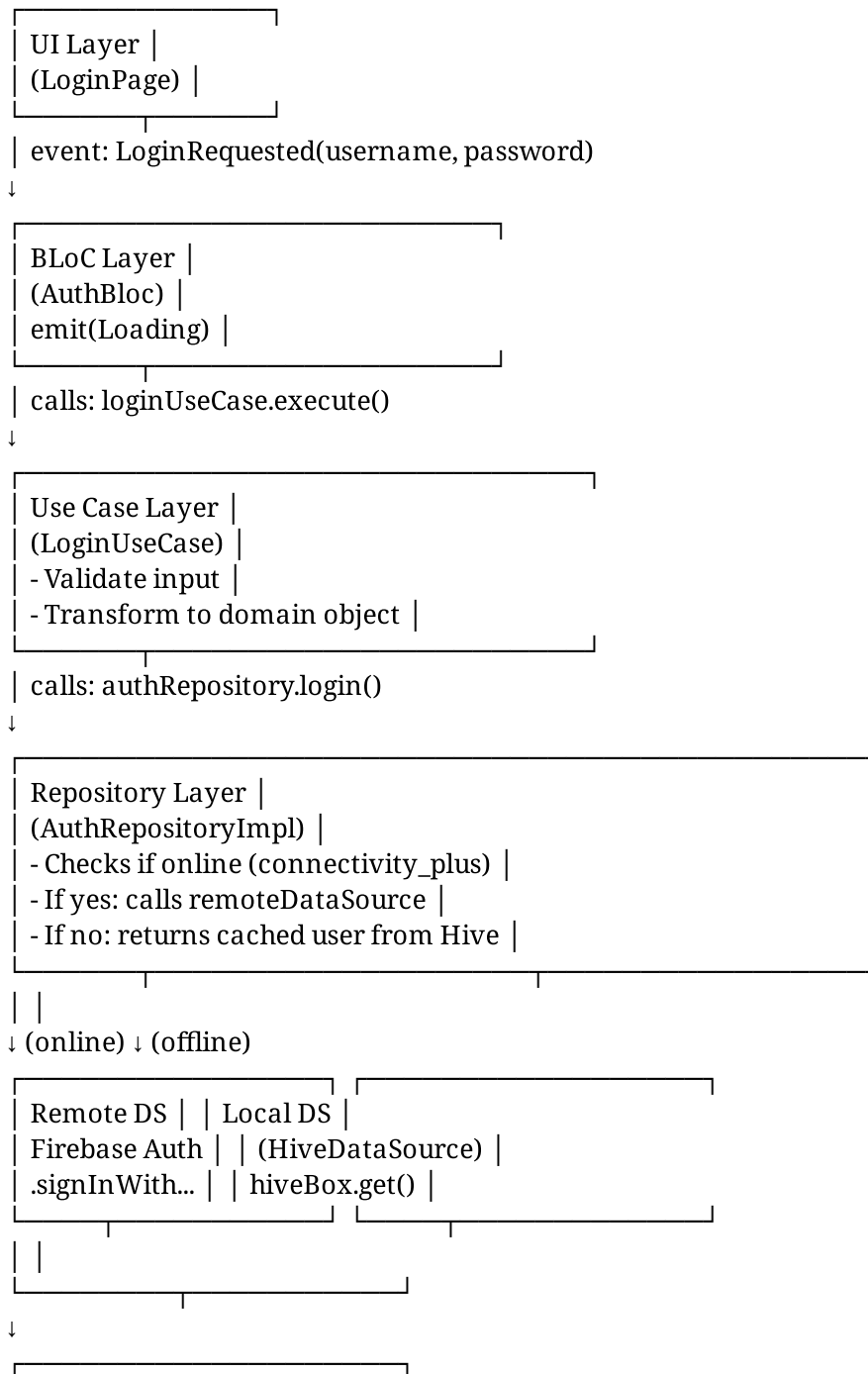
- Token security (expiry, rotation)
- Encryption verification

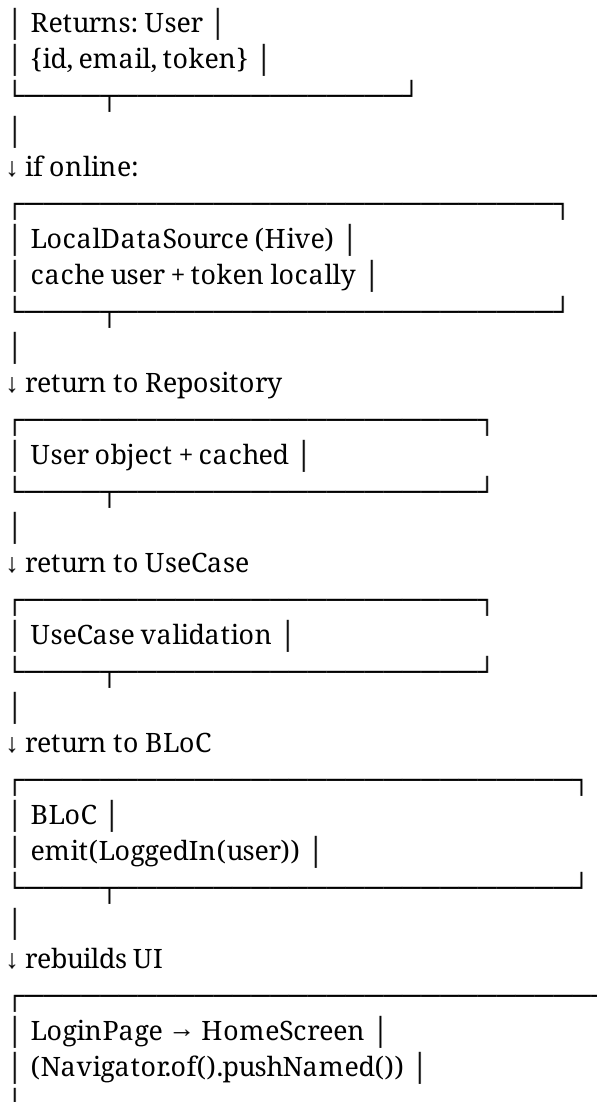
Deliverable:

- Polished, production-ready app
- Advanced features (Face ID, notifications) ready for v1.5

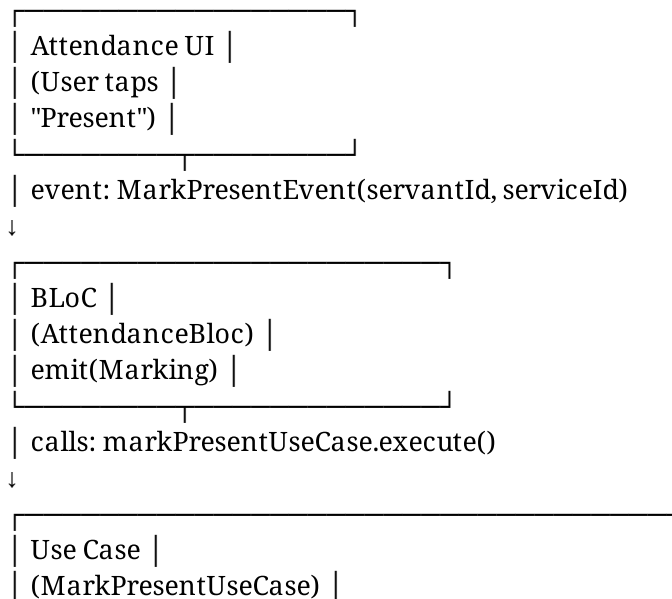
Part 6: Specific How Components Communicate

6.1 Example 1: Login Flow





6.2 Example 2: Mark Attendance (Offline-First)



| - Validate servant exists |
| - Create AttendanceRecord object |

| calls: attendanceRepository.markPresent()
↓

Repository |
(AttendanceRepositoryImpl) |
- Immediately write to Hive (local) |
- Return success to caller |
- Check if online: |
 ├ Yes: add to sync queue |
 └ No: mark as "pending_sync" |

| |
↓ (always) ↓ (if online)

Local DS (Hive) |
Create record: |
{servantId, |
status:"present" |
timestamp: now |
synced: false} |

|
↓ return immediately

BLoC receives success |
emit(Marked) |
UI shows ✓ (instant!) |

|
[Time passes, servant stays online...]

↓ (background sync manager wakes up)

SyncService (background job) |
- Checks if online |
- Gets all "pending_sync" from |
Hive (synced: false) |

| calls: attendanceRepository.syncPendingRecords()
↓

Repository |
- Diff local Hive vs Firestore |
- Handle conflicts if any |
- Upload local changes |

| calls: remoteDataSource.uploadRecords()
↓

```
Remote DS (Firestore) |  
.collection('attendance') |  
.doc(serviceId) |  
.update({records: {...}}) |
```

↓ Firestore acknowledges

```
Repository |  
- Mark all uploaded records as |  
synced: true in Hive |
```

↓ notify user (silent or toast)

```
SyncService |  
emit(SyncComplete) |  
show "✓ Synced" toast |
```

6.3 Example 3: Admin Uploads Results (Real-time Sync to All Servants)

Admin opens app:

```
Admin Dashboard |  
(click "Upload") |
```

↓ Select CSV file

```
Parse CSV: |  
servantId | ScriptureScore |  
S001 | 95 |  
S002 | 88 |
```

↓ clicks "Preview"

```
Show all data + confirm |  
(Click "Publish Term") |
```

↓ BLoC calls: publishTermUseCase

```
Cloud Function (backend) |  
- Validate data format |  
- Assign term ID (auto) |  
- For each servant: |  
  └─ Create doc: results/ |  
    {servantId}/terms/...
```

| └ Set: synced_at: now() |

|
↓ Firestore writes

| Firestore Realtime Listener |
| (active on all servants' phones) |
| TRIGGERS: |
| "New document in results/{servantId}" |

|
↓ each servant's app wakes up

| AttendanceRepository |
| - Receives Firestore event |
| - Updates local Hive cache |
| - Sends event to BLoC |

|
↓ BLoC emits: NewResultsAvailable

| UI updates automatically: |
| - Show notification |
| - Results tab badge: "New" |
| - Servant opens Results tab |
| - Sees new term + scores |
| (all from local Hive, instant) |

Part 7: Technology Stack Details

7.1 Key Dependencies

dependencies:

flutter:

sdk: flutter

Firebase

firebase_core: ^2.24.0

firebase_auth: ^4.14.0

cloud_firestore: ^4.14.0

firebase_storage: ^11.5.0

firebase_crashlytics: ^3.4.0

firebase_analytics: ^10.7.0

firebase_messaging: ^14.7.0 # For notifications (Phase 4)

Local Storage

hive: ^2.2.3
hive_flutter: ^1.1.0
hive_generator: ^2.0.1

Security

flutter_secure_storage: ^9.0.0

Networking & Offline

connectivity_plus: ^5.1.0
http: ^1.1.0
dio: ^5.3.0 # For advanced HTTP

UI

flutter_bloc: ^8.1.0
get_it: ^7.6.0 # Service locator
equatable: ^2.0.5
go_router: ^12.0.0 # Routing

Scanning

flutter_barcode_scanner: ^2.0.0

Utils

dartz: ^0.10.1 # Functional programming
intl: ^0.19.0 # Internationalization (Arabic)

dev_dependencies:
build_runner: ^2.4.6
hive_generator: ^2.0.1
bloc_test: ^9.1.0
mockito: ^5.4.2
flutter_test:
sdk: flutter
flutter_lints: ^3.0.0

7.2 Firebase Configuration

Android (google-services.json):

- Enable Firebase Auth (Email/Password)
- Enable Cloud Firestore
- Enable Cloud Storage
- Enable Cloud Messaging (for Phase 4)

iOS (GoogleService-Info.plist):

- Same as above

Firestore Rules (from Day 1):

```
rules_version = '2';
```

```
service cloud.firestore {
```

```
  match /databases/{database}/documents {
```

```
    // Servant can read/write own data only
    match /results/{servantId}/{document=**} {
      allow read: if request.auth.uid == servantId;
      allow write: if request.auth.token.role == 'admin';
    }

    match /attendance/{serviceId}/{document=**} {
      allow read: if request.auth.token.role in ['admin', 'servant'];
      allow write: if request.auth.token.role == 'admin';
    }

    // Admin only
    match /admin/{document=**} {
      allow read, write: if request.auth.token.role == 'admin';
    }

    // Audit logs (admins can read, system writes)
    match /audit_logs/{logId} {
      allow read: if request.auth.token.role == 'admin';
      allow write: if request.auth.token.role == 'admin' || request.auth != null;
    }
  }
}
```

```
}
}
```

Part 8: Real-World Problems This Solves

Problem #1: Student Privacy Violation ✓ SOLVED

Before: Anyone can guess ID → see any student's grades

After: Username + Password + Firestore Rules → only your data

How: allow read: if request.auth.uid == servantId;

Problem #2: Single Admin Dependency ✓ SOLVED

Before: Only one person can mark attendance

After: Any servant with app can mark attendance

How: Role-based permissions + distributed mobile apps

Problem #3: Offline Reliability ✓ SOLVED

Before: Weak WiFi → data loss

After: Full offline support + automatic sync

How: Hive local + Firestore cloud + conflict resolution

Problem #4: Distribution Headache ✓ SOLVED

Before: Manual APK distribution

After: Google Play → auto-updates

How: Single upload to Play Store, users get updates automatically

Problem #5: No Data Integrity ✓ SOLVED

Before: No way to verify who did what

After: Audit logs + timestamps + user IDs

How: Every action logged to Firestore

Problem #6: Multi-Terminal History Missing ✓ SOLVED

Before: Only last term visible

After: All terms + timeline visualization

How: Firestore stores all terms + Hive caches locally

Part 9: Implementation Tips & Best Practices

9.1 Development Workflow

1. **Local First:** Always test offline first (disable WiFi)
2. **Firestore Emulator:** Use Firebase emulator for testing (free, fast)
3. **Incremental Rollout:** Deploy to beta track first, then production
4. **Feature Flags:** Use Firebase Remote Config for gradual rollout
5. **Monitoring:** Watch Crashlytics + Analytics daily

9.2 Security Checklist

- ☐ Firestore rules don't allow public read/write
- ☐ JWT tokens expire every 24 hours
- ☐ Passwords never stored locally (only in secure storage)
- ☐ API keys rotated regularly
- ☐ Audit logs enabled for all sensitive operations
- ☐ SSL pinning for API (if custom backend)

9.3 Testing Strategy

Level	Tools	Example
Unit	Mockito, Test	Auth use cases, sync logic
Widget	flutter_test, WidgetTester	Login form, attendance list
Integration	firebase_emulator	End-to-end flows
Manual (on Device)	Real phone	Offline → online transition, real barcode scanning

Part 10: Next Steps (Week-by-Week Checklist)

Week 1 (Action Items)

- ☐ Review this plan with the team
- ☐ Set up Firebase project (console.firebase.google.com)
- ☐ Create GitHub repo for unified app
- ☐ Download old data from current apps (backup)
- ☐ Start Firebase + Flutter setup (code-gen, dependencies)

Week 2

- ☐ Complete authentication module
- ☐ Test login/logout flows
- ☐ Data migration script ready

Week 3

- ☐ Sync engine core logic
- ☐ Local storage (Hive) working
- ☐ Offline detection + background sync

Weeks 4–8 (Phase 1 complete)

- [] Migration done + verified
- [] Full security layer operational
- [] Team can test auth locally

Risks & Mitigation

Risk	Impact	Mitigation
Firestore data loss	Complete system failure	Daily automated backups (Cloud Storage) + Firestore point-in-time recovery
Token expiry during offline use	User stuck at login	Refresh token strategy + offline grace period (24h)
Sync conflicts	Data corruption	Manual conflict resolution UI + last-write-wins with audit trail
Face ID training delay	Phase 2 blocked	Start Phase 2 without Face ID, add later as Phase 2.5
Google Play review rejection	Launch delayed	Early submission for review (Week 20, not Week 22)
Large dataset performance	App lags	Pagination + lazy loading from day 1

Glossary

- **Hive:** Local Flutter database (fast, encrypted, works offline)
 - **Firestore:** Google's cloud database (real-time, security rules, scalable)
 - **BLoC:** Business Logic Component (state management pattern)
 - **Use Case:** Domain business logic (independent of UI/database)
 - **Repository:** Abstraction layer (combines local + remote data)
 - **Data Source:** Actual implementation (Firestore, Hive, API)
 - **Sync:** Merging local changes with cloud
 - **Conflict Resolution:** Deciding which version wins when data differs
 - **Role-Based Access:** Permissions based on user role (Admin vs Servant)
 - **Offline-First:** Works locally, syncs when online
 - **Audit Log:** Record of who did what, when, from where
-

Questions to Finalize

Before we code, ask the team:

1. **Timeline:** Is 5–6 months realistic given your schedules?
2. **Face ID:** Should we reserve the architecture for it, or skip until v1.5?
3. **Multi-language:** Arabic only, or English too?
4. **Permissions:** Which servants should see attendance reports (admin only, or all)?
5. **Export:** CSV only, or also PDF?
6. **Data retention:** How long to keep old results (1 year, 5 years, forever)?
7. **Backup strategy:** Daily automatic backup to Google Drive?
8. **Testing environment:** Can we use real servants for beta testing Week 18?
9. **Google Play:** Can you get father's approval for Play Store submission?
10. **Budget:** Any cloud costs acceptable (currently free tier, \$0/month)?

Conclusion

This unified system solves every real-world problem mentioned:

- ✓ Security (username + password + Firestore rules)
- ✓ Offline-first (works without WiFi)
- ✓ Multi-device sync (all servants can mark attendance)
- ✓ Auto-distribution (Google Play handles updates)
- ✓ Scalable (Firebase handles growth automatically)
- ✓ Auditable (every action logged)
- ✓ Maintainable (clean architecture + tests)
- ✓ Future-proof (Face ID ready, notifications ready)

Recommendation: Start Phase 1 this week. Build the foundation first (auth + sync), then features will follow naturally.

ربنا معاكم ويقويكم ♥

Appendices

A. File Structure (Full Flutter Project)

[See Part 4.2 above]

B. Firestore Security Rules (Complete)

[See Part 7.2 above]

C. Data Migration SQL Script

[Available in separate document]

D. Testing Checklist

[Available in separate document]

E. Deployment Checklist

[Available in separate document]

Document prepared for: Church Leadership, Development Team

Status: Ready for Implementation

Prepared by: Kerollos + Barthy + AI Analysis